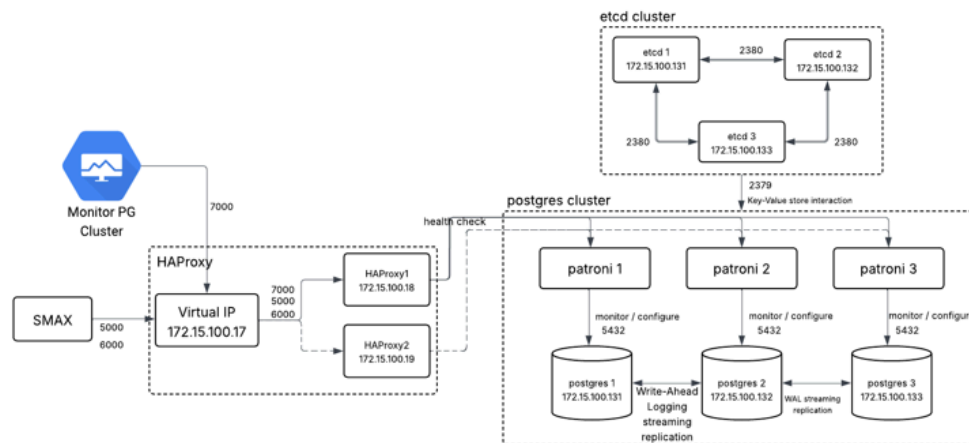


Setting Up a PostgreSQL Cluster with a Load Balancer using etcd, patroni, HAProxy and Keepalived



VM Setup

Here is our infrastructure setup

• PostgreSQL Nodes

- 172.15.100.131 - dev-postgres-1
- 172.15.100.132 - dev-postgres-2
- 172.15.100.133 - dev-postgres-3

• Patroni Cluster (runs on the same nodes as PostgreSQL)

- 172.15.100.131
- 172.15.100.132
- 172.15.100.133

• etcd Cluster

- 172.15.100.131 - etcd-1
- 172.15.100.132 - etcd-2
- 172.15.100.133 - etcd-3

• HAProxy (Load Balancer)

- 172.15.100.18
- 172.15.100.19

• Keepalived Virtual Ip

- 172.15.100.17

Prerequisites

1. Install required packages (nano and wget)

```
sudo dnf install -y nano wget
```

2. Open required firewall ports

- For PostgreSQL (5432) and etcd dashboard (8008)

```
1 sudo firewall-cmd --add-port={5432,8008}/tcp --permanent
2 sudo firewall-cmd --reload
```

- For HAProxy communication (5000,6000,7000)

```
1 sudo firewall-cmd --add-port={5000,6000,7000}/tcp --permanent
2 sudo firewall-cmd --reload
```

- For etcd internal communication (2379, 2380)

```
1 sudo firewall-cmd --add-port={2379,2380}/tcp --permanent
2 sudo firewall-cmd --reload
```

Install and Configure etcd

1. Download and install etcd RPM

```
1 wget https://ftp.postgresql.org/pub/repos/yum/common/pgdg-rhel9-
  extras/redhat/rhel-9-x86_64/etcd-3.5.11-1PGDG.rhel9.x86_64.rpm
2 sudo rpm -ihv etcd-3.5.11-1PGDG.rhel9.x86_64.rpm
```

2. Verify installation

```
1 etcd --version
```

Expected output

```
1 etcd version 3.5.11
```

3. Configure etcd

Modify the `/etc/etcd/etcd.conf` file on each etcd node as follows:

etcd1 - 172.15.100.131

```
1 [Member]
2 ETCD_LISTEN_PEER_URLS="http://172.15.100.131:2380"
3 ETCD_LISTEN_CLIENT_URLS="http://172.15.100.131:2379,http://localhost:2
  379"
4 ETCD_NAME="etcd1"
5
6 [Clustering]
7 ETCD_INITIAL_ADVERTISE_PEER_URLS="http://172.15.100.131:2380"
8 ETCD_ADVERTISE_CLIENT_URLS="http://172.15.100.131:2379"
9 ETCD_INITIAL_CLUSTER="etcd1=http://172.15.100.131:2380,etcd2=http://17
  2.15.100.132:2380,etcd3=http://172.15.100.133:2380"
10 ETCD_INITIAL_CLUSTER_TOKEN="etcd-cluster"
11 ETCD_INITIAL_CLUSTER_STATE="new"
```

etcd2 - 172.15.100.132

```
1 [Member]
2 ETCD_LISTEN_PEER_URLS="http://172.15.100.132:2380"
3 ETCD_LISTEN_CLIENT_URLS="http://172.15.100.132:2379,http://localhost:2
379"
4 ETCD_NAME="etcd2"
5
6 [Clustering]
7 ETCD_INITIAL_ADVERTISE_PEER_URLS="http://172.15.100.132:2380"
8 ETCD_ADVERTISE_CLIENT_URLS="http://172.15.100.132:2379"
9 ETCD_INITIAL_CLUSTER="etcd1=http://172.15.100.131:2380,etcd2=http://17
2.15.100.132:2380,etcd3=http://172.15.100.133:2380"
10 ETCD_INITIAL_CLUSTER_TOKEN="etcd-cluster"
11 ETCD_INITIAL_CLUSTER_STATE="new"
```

etcd3 - 172.15.100.133

```
1 [Member]
2 ETCD_LISTEN_PEER_URLS="http://172.15.100.133:2380"
3 ETCD_LISTEN_CLIENT_URLS="http://172.15.100.133:2379,http://localhost:2
379"
4 ETCD_NAME="etcd3"
5
6 [Clustering]
7 ETCD_INITIAL_ADVERTISE_PEER_URLS="http://172.15.100.133:2380"
8 ETCD_ADVERTISE_CLIENT_URLS="http://172.15.100.133:2379"
9 ETCD_INITIAL_CLUSTER="etcd1=http://172.15.100.131:2380,etcd2=http://17
2.15.100.132:2380,etcd3=http://172.15.100.133:2380"
10 ETCD_INITIAL_CLUSTER_TOKEN="etcd-cluster"
11 ETCD_INITIAL_CLUSTER_STATE="new"
```

4. Set permissions

Ensure that `/var/lib/etcd` is owned by the `etcd` user

```
1 sudo chown -R etcd:etcd /var/lib/etcd
```

5. Start and Enable etcd

Enable etcd to start on boot

```
1 sudo systemctl enable etcd
```

Start etcd service

```
1 sudo systemctl start etcd
```

Check etcd status

```
1 sudo systemctl status etcd
```

6. Validate Cluster Setup

Check etcd member list

```
1 etcdctl member list
```

Expected output:

```
1 af0ff58dea7c2fbc, started, etcd1, http://172.15.100.131:2380,  
http://172.15.100.131:2379, false  
2 b58ee750ce63bcb5, started, etcd2, http://172.15.100.132:2380,  
http://172.15.100.132:2379, false  
3 efca36c0e1a94fc6, started, etcd3, http://172.15.100.133:2380,  
http://172.15.100.133:2379, false
```

Install PostgreSQL 16 on RHEL 9

1. Add the PostgreSQL repository

```
1 sudo dnf install -y  
https://download.postgresql.org/pub/repos/yum/reposrpm/EL-9-  
x86_64/pgdg-redhat-repo-latest.noarch.rpm
```



Tip: You can find the latest `.rpm` for your desired version and platform on the official [PostgreSQL download site](https://www.postgresql.org/download/linux/redhat/).

1. Select version:

2. Select platform:

3. Select architecture:

4. Copy, paste and run the relevant parts of the setup script:

2. Disable the built-in PostgreSQL module

```
1 sudo dnf -qy module disable postgresql
```

3. Install PostgreSQL 16

```
1 sudo dnf install -y postgresql16-server
```

4. Verify installation

```
1 rpm -qa | grep postgresql16
```

Expected output

```
1 postgresql16-libs-16.8-1PGDG.rhel9.x86_64  
2 postgresql16-16.8-1PGDG.rhel9.x86_64  
3 postgresql16-server-16.8-1PGDG.rhel9.x86_64
```

Install patroni

1. Ensure you are using Python 3.9+

```
python3 --version
```

2. Install pip (if not installed)

```
sudo dnf install -y python3-pip
```

3. Upgrade pip (recommended)

```
python3 -m pip install --upgrade pip
```

4. Install required dependencies

```
pip3 install psycopg2-binary pip3 install 'patroni[etcd,consul]'
```

5. Verify Patroni installation

```
patroni --version # Should return: patroni 4.0.5
```

If you see

```
-bash: /usr/local/bin/patroni: No such file or directory
```

Add `~/.local/bin` to your PATH

```
echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc source ~/.bashrc
```

Test again

```
patroni --version
```

6. Create Configuration Directory

```
mkdir -p /usr/patroni/conf
```

7. Generate SSL Certificates

```
1 sudo openssl genrsa -out server.key 2048
2 sudo openssl req -new -x509 -days 3650 -key server.key -out server.crt
  \ -subj
  "/C=LK/ST=Western/L=Colombo/O=CloudSolutions/OU=PostgresCluster/CN=172.
  15.100.18"
3 sudo cp server.crt /usr/patroni/conf/
4 sudo cp server.key /usr/patroni/conf/
```

8. Set proper permissions

```
1 cd /usr/patroni/conf
2 sudo chmod 400 server.crt
3 sudo chmod 400 server.key
4 sudo chown postgres:postgres server.key
5 sudo chown postgres:postgres server.crt
```

9. Create Patroni Configuration Files

Create the YAML config for each node. Adjust the `name`, `listen`, and `connect_address` per node.

```
sudo nano /usr/patroni/conf/postgresql.yml
```

div-postgres-1 - 172.15.100.131

```
1 scope: postgresql-cluster
2 namespace: /pg-cluster/
3 name: postgresql-01
4
5 restapi:
6   listen: 172.15.100.131:8008
7   connect_address: 172.15.100.131:8008
8
9 etcd:
10   hosts: 172.15.100.131:2379,172.15.100.132:2379,172.15.100.133:2379
11   protocol: http
12
13 bootstrap:
14   dcs:
15     ttl: 30
16     loop_wait: 10
17     retry_timeout: 10
18     maximum_lag_on_failover: 1048576
19     synchronous_mode: false
20     postgresql:
21       use_pg_rewind: true
22       use_slots: true
23       parameters:
24         ssl: true
25         ssl_cert_file: /usr/patroni/conf/server.crt
26         ssl_key_file: /usr/patroni/conf/server.key
27         max_connections: 2000
28         shared_buffers: 4GB
29         work_mem: 16MB
30         maintenance_work_mem: 1GB
31         wal_buffers: 64MB
32         max_wal_size: 2GB
33         min_wal_size: 1GB
34         effective_cache_size: 8GB
35         fsync: on
36         checkpoint_completion_target: 0.9
37         log_rotation_size: 100MB
38         listen_addresses: '*'
39         temp_buffers: 8MB
40     pg_hba:
41       - host replication replicator 127.0.0.1/32 md5
42       - host replication replicator 172.15.100.131/32 md5
43       - host replication replicator 172.15.100.132/32 md5
44       - host replication replicator 172.15.100.133/32 md5
45       - host all all 0.0.0.0/0 md5
46     initdb:
47       - encoding: UTF8
48       - data-checksums
49
50 users:
51   admin:
52     password: admin
53     options:
54       - createrole
55       - createdb
56
57 postgresql:
58   listen: 172.15.100.131:5432
59   connect_address: 172.15.100.131:5432
60   data_dir: /var/lib/pgsql/16/data
61   bin_dir: /usr/pgsql-16/bin
62   pgpass: /tmp/pgpass
63   authentication:
64     superuser:
```

```

65     username: postgres
66     password: postgres
67     replication:
68         username: replicator
69         password: replicator
70     rewind:
71         username: pgrewind
72         password: pgrewind
73
74 tags:
75     nofailover: false
76     noloadbalance: false
77     clonefrom: false
78     nosync: true

```

div-postgres-2 - 172.15.100.132

```

1  scope: postgresql-cluster
2  namespace: /pg-cluster/
3  name: postgresql-02
4  restapi:
5      listen: 172.15.100.132:8008
6      connect_address: 172.15.100.132:8008
7  etcd:
8      hosts: 172.15.100.131:2379,172.15.100.132:2379,172.15.100.133:2379
9      protocol: http
10 bootstrap:
11     dcs:
12         ttl: 30
13         loop_wait: 10
14         retry_timeout: 10
15         maximum_lag_on_failover: 1048576
16         synchronous_mode: false
17     postgresql:
18         use_pg_rewind: true
19         use_slots: true
20         parameters:
21             ssl: true
22             ssl_cert_file: /usr/patroni/conf/server.crt
23             ssl_key_file: /usr/patroni/conf/server.key
24             max_connections: 2000
25             shared_buffers: 4GB
26             work_mem: 16MB
27             maintenance_work_mem: 1GB
28             wal_buffers: 64MB
29             max_wal_size: 2GB
30             min_wal_size: 1GB
31             effective_cache_size: 8GB
32             fsync: on
33             checkpoint_completion_target: 0.9
34             log_rotation_size: 100MB
35             listen_addresses: '*'
36             temp_buffers: 8MB
37     pg_hba:
38         - host replication replicator 127.0.0.1/32 md5
39         - host replication replicator 172.15.100.131/32 md5
40         - host replication replicator 172.15.100.132/32 md5
41         - host replication replicator 172.15.100.133/32 md5
42         - host all all 0.0.0.0/0 md5
43     initdb:
44         - encoding: UTF8
45         - data-checksums
46     users:
47         admin:
48             password: admin
49             options:

```

```

50     - createrole
51     - createdb
52 postgresql:
53     listen: 172.15.100.132:5432
54     connect_address: 172.15.100.132:5432
55     data_dir: /var/lib/pgsql/16/data
56     bin_dir: /usr/pgsql-16/bin
57     pgpass: /tmp/pgpass
58     authentication:
59         superuser:
60             username: postgres
61             password: postgres
62         replication:
63             username: replicator
64             password: replicator
65         rewind:
66             username: pgrewind
67             password: pgrewind
68 tags:
69     nofailover: false
70     noloadbalance: false
71     clonefrom: false
72     nosync: true

```

div-postgres-3 - 172.15.100.133

```

1  scope: postgresql-cluster
2  namespace: /pg-cluster/
3  name: postgresql-03
4
5  restapi:
6      listen: 172.15.100.133:8008
7      connect_address: 172.15.100.133:8008
8
9  etcd:
10     hosts: 172.15.100.131:2379,172.15.100.132:2379,172.15.100.133:2379
11     protocol: http
12
13 bootstrap:
14     dcs:
15         ttl: 30
16         loop_wait: 10
17         retry_timeout: 10
18         maximum_lag_on_failover: 1048576
19         synchronous_mode: false
20     postgresql:
21         use_pg_rewind: true
22         use_slots: true
23         parameters:
24             ssl: true
25             ssl_cert_file: /usr/patroni/conf/server.crt
26             ssl_key_file: /usr/patroni/conf/server.key
27             max_connections: 2000
28             shared_buffers: 4GB
29             work_mem: 16MB
30             maintenance_work_mem: 1GB
31             wal_buffers: 64MB
32             max_wal_size: 2GB
33             min_wal_size: 1GB
34             effective_cache_size: 8GB
35             fsync: on
36             checkpoint_completion_target: 0.9
37             log_rotation_size: 100MB
38             listen_addresses: '*'
39             temp_buffers: 8MB
40     pg_hba:

```



```

41     - host replication replicator 127.0.0.1/32 md5
42     - host replication replicator 172.15.100.131/32 md5
43     - host replication replicator 172.15.100.132/32 md5
44     - host replication replicator 172.15.100.133/32 md5
45     - host all all 0.0.0.0/0 md5
46 initdb:
47     - encoding: UTF8
48     - data-checksums
49
50 users:
51     admin:
52         password: admin
53         options:
54             - createrole
55             - createdb
56
57 postgresql:
58     listen: 172.15.100.133:5432
59     connect_address: 172.15.100.133:5432
60     data_dir: /var/lib/pgsql/16/data
61     bin_dir: /usr/pgsql-16/bin
62     pgpass: /tmp/pgpass
63     authentication:
64         superuser:
65             username: postgres
66             password: postgres
67         replication:
68             username: replicator
69             password: replicator
70     rewind:
71         username: pgrewind
72         password: pgrewind
73
74 tags:
75     nofailover: false
76     noloadbalance: false
77     clonefrom: false
78     nosync: true

```

10. Create systemd Service for Patroni

```
sudo nano /usr/lib/systemd/system/patroni.service
```

All three nodes include this

```

1 [Unit]
2 Description=patroni
3 Documentation=https://patroni.readthedocs.io/en/latest/index.html
4 After=syslog.target network.target etcd.target
5 Wants=network-online.target
6
7 [Service]
8 Type=simple
9 User=postgres
10 Group=postgres
11 PermissionsStartOnly=true
12 ExecStart=/usr/local/bin/patroni /usr/patroni/conf/postgresql.yml
13 ExecReload=/bin/kill -HUP $MAINPID
14 LimitNOFILE=65536
15 KillMode=process
16 KillSignal=SIGINT
17 Restart=on-abnormal
18 RestartSec=30s
19 TimeoutSec=0
20

```

```
21 [Install]
22 WantedBy=multi-user.target
```

11. Move and make patroni binary files executable

```
1 sudo mv /home/db-admin/.local/bin/patroni /usr/local/bin/patroni
2 sudo chmod +x /usr/local/bin/patroni
```

12. Check SELinux status

```
sestatus
```

If it says enabled, switch it to disabled

```
sudo setenforce 0
```

`setenforce 0` **only lasts until reboot.** To make SELinux stay disabled across reboots (the guide wants this), also edit the config file on **each** node.

```
1 sudo nano /etc/selinux/config
```

Change the line `SELINUX=enforcing` to `SELINUX=disabled`, save. (`setenforce 0` covers you right now; this makes it survive a reboot so Patroni/etcd don't break later.)

13. Start Patroni (on All Nodes)

These commands should be run on all nodes. The node that runs first becomes the leader of the cluster

```
1 sudo systemctl daemon-reload
2 sudo systemctl start patroni
3 sudo systemctl enable patroni
4 sudo systemctl status patroni
```

SetUp HAProxy on two hosts

1. Install HAProxy

```
1 sudo yum -y install haproxy
```

2. Back up the existing HAProxy configuration file of the HAProxy instance

```
1 cp -p /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.bkp
```

3. HAProxy configuration file

Remove the existing configuration file and create a new configuration file.

```
1 sudo rm -r /etc/haproxy/haproxy.cfg
2 sudo nano /etc/haproxy/haproxy.cfg
```

Remove the content in the file and copy the following to the file.

```
1 global
```

```

2    log 127.0.0.1 local2
3    chroot /var/lib/haproxy
4    pidfile /var/run/haproxy.pid
5    maxconn 6000
6    user haproxy
7    group haproxy
8    daemon
9    stats socket /var/lib/haproxy/stats
10 defaults
11     mode tcp
12     log global
13     retries 3
14     timeout queue 1m
15     timeout connect 10s
16     timeout client 31m
17     timeout server 31m
18     timeout check 10s
19     maxconn 3000
20 listen stats
21     mode http
22     bind *:7000
23     stats enable
24     stats uri /
25 listen postgres
26     bind *:5000
27     option httpchk
28     http-check expect status 200
29     default-server inter 3s fall 3 rise 2 on-marked-down shutdown-
sessions
30     server postgresql-01 172.15.100.131:5432 maxconn 2000 check port
8008
31     server postgresql-02 172.15.100.132:5432 maxconn 2000 check port
8008
32     server postgresql-03 172.15.100.133:5432 maxconn 2000 check port
8008
33 listen postgres-readonly
34     bind *:6000
35     option httpchk GET /replica
36     http-check expect status 200
37     default-server inter 3s fall 3 rise 2 on-marked-down shutdown-
sessions
38     server postgresql-01 172.15.100.131:5432 maxconn 2000 check port
8008
39     server postgresql-02 172.15.100.132:5432 maxconn 2000 check port
8008
40     server postgresql-03 172.15.100.133:5432 maxconn 2000 check port
8008

```

4. Reload the system daemon

```
1 | sudo systemctl daemon-reload
```

5. Make sure the SELinux boolean "haproxy_connect_any" is on

```
1 | getsebool -a|grep haproxy_connect
```

If the result is like

```
1 | haproxy_connect_any --> off
```

Then, run the following command

```
1 | setsebool -P haproxy_connect_any=1
```

6. Start the HAProxy service

```
1 systemctl start haproxy
2 systemctl status haproxy
```

Enable HAProxy to run at startup

```
1 systemctl enable haproxy
```

7. Check the connection

Make sure you can connect to PostgreSQL Server through HAProxy by running the following commands on a host where "psql" is available. If you want to access the cluster from a local machine, try the command using that machine.

```
1 psql -h 172.15.100.18 -p 5000 -d postgres -U postgres
2 psql -h 172.15.100.19 -p 5000 -d postgres -U postgres
```

Then log into any of the 172.15.100.131 / 132 / 133 nodes and copy the certificate from this path /usr/patroni/conf/server.crt .Then add this certificate to a server.crt file and locate it on your local machine.

Use an SSL connection to the database

```
1 psql "host=172.15.100.18 port=5000 dbname=postgres user=postgres
password=postgres sslmode=verify-ca sslrootcert=<your local path to
server.crt certificate path>"
2 psql "host=172.15.100.19 port=5000 dbname=postgres user=postgres
password=postgres sslmode=verify-ca sslrootcert=<your local path to
server.crt certificate path>"
```

Check if the connections to the different ports are in read-only mode.

```
1 psql -h 172.15.100.18 -U postgres -p 6000 -c "SELECT
pg_is_in_recovery();"

```

Expected output for the readonly port

```
1 pg_is_in_recovery
2 -----
3 t
4 (1 row)
```

Check if the connections to the different ports are in write-read mode.

```
1 psql -h 172.15.100.18 -U postgres -p 5000 -c "SELECT
pg_is_in_recovery();"

```

Output for the write-read port

```
1 pg_is_in_recovery
2 -----
3 f
4 (1 row)
```

try with 172.15.100.19 commands as well.

Test the Postgres Cluster

1. Test and verify PostgreSQL cluster

by initiating a connection request to HAProxy

```
1 | psql -h 172.15.100.18 -p 5000 -d postgres -U postgres
```

2. Open a web browser and access http://172.15.100.18:7000

HAProxy version 2.4.22-f8e3218, released 2023/02/14

Statistics Report for pid 843

> General process information

pid = 843 (process v1, nproc = 1, nbthread = 2)
uptime = 50 (969m07s)
system limits: memmax = unlimited, ulimit.s = 12043
maxsocks = 12043, maxconn = 6000, maxusers = 0
current conn = 2, current pjobs = 60, conn rate = 1/sec, bit rate = 0.271 kbps
Running tasks: 0/0, size = 100 %

active UP
active UP: going down
active DOWN: going up
active or backup DOWN
active or backup SOFT STOPPED for maintenance (MAINT)
Note: "NOBODYDRARY" = UP with load balancing disabled

backup UP
backup UP: going down
backup DOWN: going up
not checked
active or backup DOWN for maintenance (MAINT)
active or backup SOFT STOPPED for maintenance

Display options:

Scope:

Hide DOWN servers

Refresh rate

Color server

JSON output (external)

External resources:

Patroni site

Patroni Wiki

Online manual

stats		Queue		Session rate		Sessions		Bytes		Denied		Errors		Warnings		Status		Server		
Frontend	Backend	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Cur	Max	Cur	Max	Cur	Max	Cur	Max
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

postgres		Queue		Session rate		Sessions		Bytes		Denied		Errors		Warnings		Status		Server		
Frontend	Backend	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Cur	Max	Cur	Max	Cur	Max	Cur	Max
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

postgres.replica		Queue		Session rate		Sessions		Bytes		Denied		Errors		Warnings		Status		Server		
Frontend	Backend	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Cur	Max	Cur	Max	Cur	Max	Cur	Max
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

To list the nodes from the command line on DB hosts, run a database failover test

```
1 | patronictl -c /usr/patroni/conf/postgresql.yml list
```

```
+ Cluster: postgresql-cluster (7496463976803160708) -----+
| Member          | Host          | Role    | State    | TL | Lag in MB | Tags          |
+-----+-----+
| postgresql-01   | 172.15.100.131 | Leader  | running  | 1  |           | nosync: true |
| postgresql-02   | 172.15.100.132 | Replica | streaming | 1  |           | nosync: true |
| postgresql-03   | 172.15.100.133 | Replica | streaming | 1  |           | nosync: true |
+-----+-----+
```

```
1 | patronictl -c /usr/patroni/conf/postgresql.yml failover
```

```
Current cluster topology
+ Cluster: postgresql-cluster (7496463976803160708) -----+
| Member          | Host          | Role    | State    | TL | Lag in MB | Tags          |
+-----+-----+
| postgresql-01   | 172.15.100.131 | Leader  | running  | 1  |           | nosync: true |
| postgresql-02   | 172.15.100.132 | Replica | streaming | 1  |           | nosync: true |
| postgresql-03   | 172.15.100.133 | Replica | streaming | 1  |           | nosync: true |
+-----+-----+
Candidate ['postgresql-02', 'postgresql-03'] []:
```

If this shows the states of the members like start failed like this

```
[db-admin@dev-postgres-2 ~]$ patronictl -c /usr/patroni/conf/postgresql.yml list
+ Cluster: postgresql-cluster (7496463976803160708) -----+
| Member          | Host          | Role    | State    | TL | Lag in MB | Tags          |
+-----+-----+
| postgresql-01   | 172.15.100.131 | Leader  | running  | 8  |           | nosync: true |
| postgresql-02   | 172.15.100.132 | Replica | start failed |   | unknown | nosync: true |
| postgresql-03   | 172.15.100.133 | Replica | start failed |   | unknown | nosync: true |
+-----+-----+
```

try to reinitialize the postgres members

```
1 patronictl -c /usr/patroni/conf/postgresql.yml reinit postgresql-  
cluster postgresql-02  
2 patronictl -c /usr/patroni/conf/postgresql.yml reinit postgresql-  
cluster postgresql-03
```

```
[db-admin@dev-postgres-3 ~]$ patronictl -c /usr/patroni/conf/postgresql.yml list  
+ Cluster: postgresql-cluster (7496463976803160708) -----+  
| Member      | Host           | Role   | State   | TL | Lag in MB | Tags          |  
+-----+  
| postgresql-01 | 172.15.100.131 | Leader | running | 8 |          | nosync: true |  
| postgresql-02 | 172.15.100.132 | Replica | streaming | 8 | 0 | nosync: true |  
| postgresql-03 | 172.15.100.133 | Replica | streaming | 8 | 0 | nosync: true |  
+-----+
```

Install Keepalived

1. Turn off SELinux

2. Modify the file "/etc/selinux/config"

Check SE status

```
1 sestatus
```

Output will be

```
1 SELinux status:                enabled  
2 SELinuxfs mount:              /sys/fs/selinux  
3 SELinux root directory:       /etc/selinux  
4 Loaded policy name:            targeted  
5 Current mode:                  enforcing  
6 Mode from config file:         disabled  
7 Policy MLS status:             enabled  
8 Policy deny_unknown status:    allowed  
9 Memory protection checking:    actual (secure)  
10 Max kernel policy version:    33
```

Changing "SELINUX=enforcing" to "SELINUX=disabled".

```
1 sudo nano /etc/selinux/config
```

3. Add rules to direct.xml file

Get the network interface name

```
1 ip -o link show | awk -F': ' '{state UP/ {print $2}'
```

Output will be

```
1 ens33
```

Back up the file "/etc/firewalld/direct.xml" if it exists.

```
1 [ -f /etc/firewalld/direct.xml ] && cp /etc/firewalld/direct.xml  
/etc/firewalld/direct.xml.bak
```

Add direct rules by running the following commands.

```
1 sudo firewall-cmd --direct --permanent --add-rule ipv4 filter INPUT 0 -
  --in-interface ens33 --destination 224.0.0.18 --protocol vrrp -j ACCEPT
2 sudo firewall-cmd --direct --permanent --add-rule ipv4 filter OUTPUT 0
  --out-interface ens33 --destination 224.0.0.18 --protocol vrrp -j
  ACCEPT
3 sudo firewall-cmd --reload
```

It will add these rules to `/etc/firewalld/direct.xml` file.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <direct>
3   <rule ipv="ipv4" table="filter" chain="INPUT" priority="0">--in-
  interface ens33 --destination 224.0.0.18 --protocol vrrp -j
  ACCEPT</rule>
4   <rule ipv="ipv4" table="filter" chain="OUTPUT" priority="0">--out-
  interface ens33 --destination 224.0.0.18 --protocol vrrp -j
  ACCEPT</rule>
5 </direct>
```

Run the following command to make sure the rules above are added

```
1 sudo firewall-cmd --direct --get-rules ipv4 filter OUTPUT
2 sudo firewall-cmd --direct --get-rules ipv4 filter INPUT
```

4. Install and configure Keepalived on both hosts.

Install keepalived in both hosts

```
1 yum install keepalived -y
```

In both hosts remove the existing configuration file and create an empty file. If you check it there will be many more configurations, but this moment we don't need that much.

```
1 cat /etc/keepalived/keepalived.conf
2 sudo rm -r /etc/keepalived/keepalived.conf
3 sudo nano /etc/keepalived/keepalived.conf
```

Put these configurations

haproxy-1 - 172.15.100.18

```
1 global_defs {
2   router_id gd_ha01
3   default_interface ens33
4 }
5 vrrp_script chk_haproxy {
6   script "killall -0 haproxy"
7   interval 2
8   fall 2
9   rise 2
10 }
11 vrrp_instance VI_1 {
12   interface ens33           # interface to monitor
13   nopreempt
14   state BACKUP              # use nopreempt, so BACKUP on both of
  haproxy1 and haproxy2
15   virtual_router_id 91      # use a unique id shared between
  haproxy1 and haproxy2
16   priority 101              # 101 on haproxy1, 100 on haproxy2
```

```

17  advert_int 1
18
19  unicast_src_ip 172.15.100.18
20  unicast_peer {
21      172.15.100.19
22  }
23  authentication {
24      auth_type PASS
25      auth_pass lb-user # specify the same password for haproxy1 and
haproxy2
26  }
27  virtual_ipaddress {
28      172.15.100.17
29  }
30  track_script {
31      chk_haproxy
32  }
33 }

```

haproxy-1 - 172.15.100.19

```

1  global_defs {
2      router_id gd_ha02
3      default_interface ens33
4  }
5  vrrp_script chk_haproxy {
6      script "killall -0 haproxy" # check the haproxy process
7      interval 2                  # every 2 seconds
8      fall 2
9      rise 2
10 }
11 vrrp_instance VI_1 {
12     interface ens33              # interface to monitor
13     nopreempt
14     state BACKUP                 # use nopreempt, so BACKUP on both of
haproxy1 and haproxy2
15     virtual_router_id 91         # use a unique id shared between
haproxy1 and haproxy2
16     priority 100                 # 101 on haproxy1, 100 on haproxy2
17     advert_int 1
18     authentication {
19         auth_type PASS
20         auth_pass lb-user # specify the same password for haproxy1 and
haproxy2
21     }
22     unicast_src_ip 172.15.100.19
23     unicast_peer {
24         172.15.100.18
25     }
26     virtual_ipaddress {
27         172.15.100.17
28     }
29     track_script {
30         chk_haproxy
31     }
32 }

```

5. configure IP forwarding and non-local binding.

Apply these commands to two hosts.

Append the following lines to the end of the `/etc/sysctl.conf` file.

```

1  net.ipv4.ip_forward = 1

```



```
2 net.ipv4.ip_nonlocal_bind = 1
```

Load the configuration

```
1 sudo sysctl -p
```

Start the Keepalived service

```
1 sudo systemctl enable keepalived
2 sudo systemctl start keepalived
```

Check the virtual IP by running the following commands on both HAProxy hosts.

```
1 sudo ip addr show dev ens33
```

The virtual IP address only appears on one of the two HAProxy hosts in the command output.

6. Change the bind address to the virtual IP for the HAProxy server.

Change the haproxy.cfg file.

```
1 sudo nano /etc/haproxy/haproxy.cfg
```

Change the "*" to the 172.15.100.17 in the following lines and save it

```
1 ...
2 bind *:7000
3 ...
4 bind *:5000
5 ...
6 bind *:6000
7 ...
```

Restart the HAProxy service

```
1 haproxy -c -f /etc/haproxy/haproxy.cfg
2 sudo systemctl stop haproxy
3 sudo systemctl start haproxy
```

7. Check the connection

Make sure you can connect to PostgreSQL Server through HAProxy by running the following commands on a host where "psql" is available. If you want to access the cluster from a local machine, try the command using that machine.

```
1 psql -h 172.15.100.18 -p 5000 -d postgres -U postgres
```

Then log into any of the 172.15.100.131 / 132 / 133 nodes and copy the certificate from this path /usr/patroni/conf/server.crt .Then add this certificate to a server.crt file and locate it on your local machine.

Use an SSL connection to the database

```
1 | psql "host=172.15.100.17 port=5000 dbname=postgres user=postgres
password=postgres sslmode=verify-ca sslrootcert=<your local path to
server.crt certificate path>"
```

Check if the connections to the different ports are in read-only mode.

```
1 | psql -h 172.15.100.17 -U postgres -p 6000 -c "SELECT
pg_is_in_recovery();"

```

Expected output for the readonly port

```
1 | pg_is_in_recovery
2 | -----
3 | t
4 | (1 row)
```

Check if the connections to the different ports are in write-read mode.

```
1 | psql -h 172.15.100.17 -U postgres -p 5000 -c "SELECT
pg_is_in_recovery();"

```

Output for the write-read port

```
1 | pg_is_in_recovery
2 | -----
3 | f
4 | (1 row)
```

Test the Keepalive service for HAProxy

1. Verify the virtual IP

Run this command in both hosts

```
1 | ip addr show dev ens33
```

ha-proxy-1 Output

```
[lb-user@ha-proxy ~]$ ip addr show dev ens33
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:43:af:5f brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 172.15.100.18/24 brd 172.15.100.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet 172.15.100.17/32 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe43:af5f/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
[lb-user@ha-proxy ~]$
```

ha-proxy-2 Output

```
[lb-user@ha-proxy-2 ~]$ ip addr show dev ens33
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c7:51:05 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 172.15.100.19/24 brd 172.15.100.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fec7:5105/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
[lb-user@ha-proxy-2 ~]$
```

2. Check the Keepalived service log

In both hosts run this command

```
1 journalctl -u keepalived -f
```

Or

```
1 journalctl -u keepalived
```

ha-proxy-1 Output

```
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: (VI_1) Backup received priority 0 advertisement
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: (VI_1) Backup received priority 0 advertisement
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: (VI_1) Receive advertisement timeout
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: (VI_1) Entering MASTER STATE
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: (VI_1) setting VIPs.
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: (VI_1) Sending/queueing gratuitous ARPs on ens33 for 172.15.100.17
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:38 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:43 ha-proxy Keepalived_vrrp[2769]: (VI_1) Sending/queueing gratuitous ARPs on ens33 for 172.15.100.17
Apr 25 12:35:43 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:43 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
```

The Keepalived service on this host is "Entering MASTER STATE."

The gratuitous ARP is sent for the virtual IP (176.16.100.17).

ha-proxy-2 Output

```
Apr 24 17:05:48 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 24 17:05:48 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 24 17:05:48 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 24 17:05:48 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:32:33 ha-proxy-2 Keepalived_vrrp[2569]: Script 'chk_haproxy' now returning 1
Apr 25 12:32:35 ha-proxy-2 Keepalived_vrrp[2569]: VRRP_Script(chk_haproxy) failed (exited with status 1)
Apr 25 12:32:35 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) Entering FAULT STATE
Apr 25 12:32:35 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) sent 0 priority
Apr 25 12:32:35 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) removing VIPs.
Apr 25 12:33:15 ha-proxy-2 Keepalived_vrrp[2569]: Script 'chk_haproxy' now returning 0
Apr 25 12:33:17 ha-proxy-2 Keepalived_vrrp[2569]: VRRP_Script(chk_haproxy) succeeded
Apr 25 12:33:17 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) Entering BACKUP STATE
Apr 25 12:33:17 ha-proxy-2 Keepalived_vrrp[2569]: VI_1: sending gratuitous ARP for 172.15.100.19
Apr 25 12:33:17 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.19
```

The Keepalived service on this host is "Entering BACKUP STATE."

The VIP is removed.

Simulate HAProxy Service problem

The virtual IP will switch to the BACKUP when the HAProxy on the MASTER is shut down.

```
1 sudo systemctl stop haproxy
2 sudo journalctl -u keepalived -f
```

ha-proxy-1 Output

```
[lb-user@ha-proxy ~]$ journalctl -u keepalived -f
Apr 25 12:35:43 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:43 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:43 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:43 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 12:35:43 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 13:09:23 ha-proxy Keepalived_vrrp[2769]: Script 'chk_haproxy' now returning 1
Apr 25 13:09:25 ha-proxy Keepalived_vrrp[2769]: VRRP_Script(chk_haproxy) failed (exited with status 1)
Apr 25 13:09:25 ha-proxy Keepalived_vrrp[2769]: (VI_1) Entering FAULT STATE
Apr 25 13:09:25 ha-proxy Keepalived_vrrp[2769]: (VI_1) sent 0 priority
Apr 25 13:09:25 ha-proxy Keepalived_vrrp[2769]: (VI_1) removing VIPs.
```

ha-proxy-2 Output

```
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) Backup received priority 0 advertisement
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) Backup received priority 0 advertisement
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) Receive advertisement timeout
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) Entering MASTER STATE
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) setting VIPs.
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: (VI_1) Sending/queueing gratuitous ARPs on ens33 for 172.15.100.17
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.17
Apr 25 13:06:23 ha-proxy-2 Keepalived_vrrp[2569]: Sending gratuitous ARP on ens33 for 172.15.100.17
```

The virtual IP will not switch back to haproxy1 when the HAProxy goes alive on it. Nopreempt will work. It will avoid the unnecessary connection switch.

On the ha-proxy-1, run the following command

```
1 | sudo systemctl start haproxy
2 | sudo journalctl -u keepalived -f
```

```
Apr 25 13:09:25 ha-proxy Keepalived_vrrp[2769]: (VI_1) sent 0 priority
Apr 25 13:09:25 ha-proxy Keepalived_vrrp[2769]: (VI_1) removing VIPs.
Apr 25 13:14:41 ha-proxy Keepalived_vrrp[2769]: Script 'chk_haproxy' now returning 0
Apr 25 13:14:43 ha-proxy Keepalived_vrrp[2769]: VRRP_Script(chk_haproxy) succeeded
Apr 25 13:14:43 ha-proxy Keepalived_vrrp[2769]: (VI_1) Entering BACKUP STATE
Apr 25 13:14:43 ha-proxy Keepalived_vrrp[2769]: VI_1: sending gratuitous ARP for 172.15.100.18
Apr 25 13:14:43 ha-proxy Keepalived_vrrp[2769]: Sending gratuitous ARP on ens33 for 172.15.100.18
```

Once those two psql commands return **f** and **t**, **the build is complete**. Then I'd strongly recommend the one test we haven't done - the **database** failover (as opposed to the HAProxy failover you've been testing)

```
1 | # on any DB node (assuming postgresql-01)
2 | patronictl -c /usr/patroni/conf/postgresql.yml switchover
```

Here's what to enter at each prompt

Primary [postgresql-01]: → just press **Enter** (accepts the default, postgresql-01).

Next it will ask Candidate ['postgresql-02', 'postgresql-03'] []: → type one of them, e.g. postgresql-02, and press Enter. This is the node that will become the new leader.

Then When should the switchover take place (e.g. 2026-07-01T12:00)

[now]: → press **Enter** for now.

Then it shows the topology and asks Are you sure you want to switchover cluster postgresql-cluster, demoting current leader postgresql-01? [y/N]: → type **y** and Enter.

It'll take a few seconds — postgresql-01 demotes to replica, postgresql-02 promotes to leader.

After it completes, verify

```
1 | patronictl -c /usr/patroni/conf/postgresql.yml list
```

Confirm HAProxy followed the new leader. From your laptop

```
1 | psql -h 172.15.100.17 -p 5000 -d postgres -U postgres -c "SELECT
```

- `pg_is_in_recovery` should still be `f` (5000 always routes to the leader)
- `inet_server_addr` should now show **172.15.100.132** (postgresql-02) instead of 172.15.100.131

Now you can view the stats page using <http://172.15.100.17:7000>

[illegible]